

How to Extract the Last n Characters of a String in Bash

Last updated: September 13, 2023



Written by:
Jimmy Azar

Scripting

awk cut Python sed tail

1. Overview

String manipulation is an essential aspect of shell scripting. The GNU Coreutils package in Bash along with `sed` and `awk` provides : In this tutorial, we'll explore how to extract the last n characters of a string in Bash.

2. Sample Task

Let's suppose we have data in a file named `data.txt`:

```
$ cat data.txt
A1B2C3RST
D4E5F6UVW
G7H8I9XYZ
CL
```

Here, `cat` shows us that the contents of `data.txt` comprise lines of capital letters intertwined with digits.

Our objective is to extract the last three characters of each line. Importantly, **if a line contains less than three characters, any existing characters should be preserved**. Let's explore different methods for solving the task.

3. Using `rev` and `cut`

The `rev` command reverses the order of the characters in every line of a given file. Therefore, **we can use `rev` in conjunction with `cut`**.

```
$ rev data.txt | cut -c 1-3 | rev
RST
UVW
XYZ
CL
```

The procedure consists of three steps:

1. *rev* reverses the order of the characters in each line
2. *cut* extracts the first three characters in each line by using the *-c* option and the *1-3* range
3. apply *rev* again to re-order the sequence of the extracted characters

As a result, we get the last three characters in their proper order.

4. Using *sed*

Alternatively, we can use *sed* to extract the last three characters using a [regular expression](#):

```
$ sed -E 's/.*(..)/\1/' data.txt
RST
UVW
XYZ
CL
```

In this case, we use the *-E* option with *sed* to enable [extended regex](#). The regular expression pattern consists of zero or more characters followed by two characters in parentheses. Finally, the entire pattern is replaced by the captured group, represented by `\1`. This way, we replace each line with its last three characters. Notably, **if a line consists of less than three characters, no match is found, and *sed* doesn't modify that line.**

5. Using *awk*

Another approach is to use the built-in [substr\(\)](#) function in *awk* to extract the required substring in each line:

```
$ awk '{ print substr($0, length($0)-2) }' data.txt
RST
UVW
XYZ
CL
```

Each line of the input file is represented by `$0`. In the *substr()* function, we specify the start index of the substring as `length($0)-2`. Therefore, we obtain the last three characters in each line.

6. Using *tail*

We can also use the [tail](#) command with the *-c* option to specify the last three characters of a string.

However, in this case, we need to iterate over each line of the file before we use the *tail* command:

```
$ cat last_three_characters.sh
#!/usr/bin/env bash
while read -r line; do
    echo -n "$line" | tail -c 3
    echo
done < data.txt
```

In the *last_three_characters.sh* script above, we use a [while loop](#) to iterate over the lines of the *data.txt* file which is provided as input. Within the *while* loop, we perform a series of steps:

1. [read](#) each line of the file and save it in the *line* variable
2. [echo](#) the *line* variable using the *-n* option to avoid the trailing [newline](#) character at the end of each line
3. [pipe](#) the result to the *tail -c 3* command to extract only the last three characters
4. [echo](#) a newline character so that each result ends up on a separate line

Notably, it's common practice to use the *-r* option with *read* in the first step to interpret [backslash characters](#) literally, in case it's needed. Next, we [grant the script execute permissions](#) via *chmod*:

```
$ chmod +x last_three_characters.sh
```

Finally, we run the script:

```
$ ./last_three_characters.sh
RST
UVW
XYZ
CL
```

Again, we see that we obtained the expected result.

7. Using Python

Another approach for solving the task is to use the [Python interpreter](#) to extract the required substring from each line:

```
$ cat last_three_characters.sh
#!/usr/bin/env bash
while read -r line; do
    echo "$line" | python3 -c "print(input()[-3:])"
done < data.txt
```

In the modified script, we still pass the *data.txt* file via stdin to a *while* loop that iterates over each line of the file. We also read each line. To do so, we echo the *line* variable and pipe the result to the Python interpreter. The *-c* option with the *python3* command enables us to execute a single command. In particular, we use the *input()* function in Python to read the result from stdin. Then, we print the last three characters as indicated. Next, we run the script:

```
$ ./last_three_characters.sh
RST
UVW
XYZ
CL
```

We see that we obtain the same result as before.

8. Conclusion

In this article, we explored different methods for extracting the last *n* characters of a string. In particular, one method included using Python. Comments are closed on this article!